

#중급반 1주차

시간 복잡도 & 누적 합

<Time Complexity & Prefix Sum>

^{2p} / <시간 복잡도>

^{16p} / <누적 합>

^{27p} / <필수 문제>

^{28p} / <연습 문제 도전>

#시간 복잡도

<Time Complexity>

알고리즘의 성능을 평가하는 지표 중 하나이자,
문제를 해결하는 데 걸리는 시간과 입력의 관계를 나타내는 함수

#시간 복잡도

<Time Complexity> - 계산하는 방법

1. (PS 한정) 최악의 경우를 가정한다.
2. 사칙연산, 대입 연산 등의 기본 연산이 수행되는 횟수를 대략적으로 계산한다.
3. 최고차항의 차수를 확인한다. (10p에서 설명)

PS = Problem Solving, 알고리즘 문제 풀이

알고리즘 대회에서 접하는 문제들은 최악의 경우만 가정해도 되지만,

전공 과목이나 연구에서는 다양한 상황을 고려합니다.

#시간 복잡도

<Time Complexity> - 계산 예시

5 N 찍기 (BOJ #2741)

<문제 설명>

- 자연수 N이 주어짐
- 1부터 N까지 한 줄에 하나씩 출력하기

<제약 조건>

- $1 \leq N \leq 100,000$

```
#include <iostream>
using namespace std;
int main() {
    int N;
    cin >> N;
    for(int i=1; i <= N; i++){
        cout << i << '\n';
    }
}
```

#시간 복잡도

<Time Complexity> - 계산 예시

5 N 찍기 (BOJ #2741)

<연산 횟수 계산>

- 변수 N을 선언하고 입력받음 -> 2번
- for문으로 i=1부터 i=N까지 더해줌 -> N+1번
- For문 내에서 출력 사용 -> N번
- 연산 횟수 $T(N) = 2N+3$

```
#include <iostream>
using namespace std;
int main() {
    int N;
    cin >> N;
    for(int i=1; i <= N; i++){
        cout << i << '\n';
    }
}
```

#시간 복잡도

<Time Complexity> - 계산 예시

5 별 찍기 1 (BOJ #2438)

<문제 설명>

- 자연수 N이 주어짐
- 첫째 줄에는 별 1개, 둘째 줄에는 별 2개, ...
- N번째 줄에는 별 N개를 찍는 문제

<제약 조건>

- $1 \leq N \leq 100$

```
#include <iostream>
using namespace std;

int main() {
    int N;
    cin >> N;
    for(int i=1; i <= N; i++){
        for(int j=1; j <= i; j++){
            cout << '*';
        }
        cout << '\n';
    }
}
```

#시간 복잡도

<Time Complexity> - 계산 예시

5 별 찍기 1 (BOJ #2438)

<연산 횟수 계산>

- 변수 N을 선언하고 입력받음 -> 2번
- i=1에서 i=N까지 더해줌 -> N+1번
- 첫번째 For문 내에서 일어나는 연산 -> $2*i+2$ 번
- 연산 횟수 $T(N) = N + 3 + \sum(2i + 2) = N^2 + 4N + 3$

```
#include <iostream>
using namespace std;

int main() {
    int N;
    cin >> N;
    for(int i=1; i <= N; i++){
        for(int j=1; j <= i; j++){
            cout << '*';
        }
        cout << '\n';
    }
}
```

#시간 복잡도

<Time Complexity> - 계산 예시

2 소수 찾기 (BOJ #1978)

<문제 설명>

- N개의 자연수가 주어짐
- 주어진 N개의 수 중 소수의 개수를 구하기

<제약 조건>

- $1 \leq N \leq 100$
- 주어지는 수는 1,000 이하 자연수

```
#include <iostream>
using namespace std;

int isPrime(int A) {
    if(A == 1) return 0;
    for(int i=2; i*i <= A; i++){
        if(A % i == 0) return 0;
    }
    return 1;
}

int main() {
    int N, ans=0;
    cin >> N;
    for(int i=0; i < N; i++){
        int x; cin >> x;
        ans += isPrime(x);
    }
    cout << ans;
}
```


#시간 복잡도

<Time Complexity> - 계산 예시

<연산 횟수 계산>

- isPrime() 함수의 연산 횟수는 약 $\sqrt{A}$ 번
- 주어진 자연수를 A라 하면,
- 연산 횟수 $T(N) = \sum \sqrt{A} \leq N\sqrt{A}$
- A의 최댓값은 1,000이므로, $T(N) = \sqrt{1000} N$

2 소수 찾기 (BOJ #1978)

```
#include <iostream>
using namespace std;

int isPrime(int A) {
    if(A == 1) return 0;
    for(int i=2; i*i <= A; i++){
        if(A % i == 0) return 0;
    }
    return 1;
}

int main() {
    int N, ans=0;
    cin >> N;
    for(int i=0; i < N; i++){
        int x; cin >> x;
        ans += isPrime(x);
    }
    cout << ans;
}
```

#시간 복잡도

<Time Complexity> - 점근 표기법

함수에서 가장 중요한 정보: **최고차항의 차수**

- $f(x) = 3x^2 + 10$, $g(x) = 100x$

- $f(1) = 13$

$g(1) = 100$

- $f(10) = 310$

$g(10) = 1,000$

- $f(100) = 30,010$

$g(100) = 10,000$

- $f(1000) = 3,000,010$

$g(1000) = 100,000$

- x 가 **특정 수보다 크면**, **항상 $f(x) > g(x)$ 를 만족함**

#시간 복잡도

<Time Complexity> - 점근 표기법

시간 복잡도는 대개 Big-O 표기법을 사용해서 표기합니다.

- $f(n) \in O(g(n))$
 - 어떤 양수 n_0 와 c 가 존재하여
 - $n \geq n_0$ 을 만족하는 모든 n 에 대해
 - $0 \leq f(n) \leq cg(n)$ 을 만족한다.
- ..?

#시간 복잡도

<Time Complexity> - 점근 표기법

- $f(n) \in O(g(n))$
 - 어떤 양수 n_0 와 c 가 존재하여 $n \geq n_0$ 을 만족하는 모든 n 에 대해
 - $0 \leq f(n) \leq cg(n)$ 을 만족한다.
- $n \geq n_0$ 을 만족하는 모든 n
 - n 이 **한 없이 커지면** 항상 부등호 성립
- 어떤 양수 c
 - 최고차항 **계수 무시** & 최고차항의 **차수만 확인**

#시간 복잡도

<Time Complexity> - 점근 표기법

1. $f(n) = 2n + 3, g(n) = n$

- $n_0 = 3, c = 3$ 이면, $2n + 3 \leq 3n : 2n + 3 \in O(n)$

2. $f(n) = n^2 + 4n + 3, g(n) = n^2$

- $n_0 = 5, c = 2$ 이면, $n^2 + 4n + 3 \leq 2n^2 : n^2 + 4n + 3 \in O(n^2)$

Big-O Notation의 의미는 $f(x)$ 가 아무리 빨리 증가해도,
“최대” $g(x)$ 에 비례하는 수준으로 증가한다는 것입니다.
따라서 $f(n)=n+1$ 역시 $O(n^2)$ 이라 쓸 수 있어요!

#시간 복잡도

<Time Complexity> - 왜 중요한가?

간단한 연산(+, = 등)은 1초에 10억 번 정도 실행되고,
복잡한 연산(% , sqrt() 등)은 1초에 1억 번 정도 실행됨

- Online Judge 시스템 기준으로, 사용 환경에 따라 크게 다를 수 있음
- 대부분의 상황에선 연산 1억 번 = 1초 라고 기억해도 일반적으로 무방함

문제 조건이 주어질 때, 주어진 시간 안에 프로그램이 실행되려면
어떤 시간 복잡도가 필요한지 가늠할 수 있음

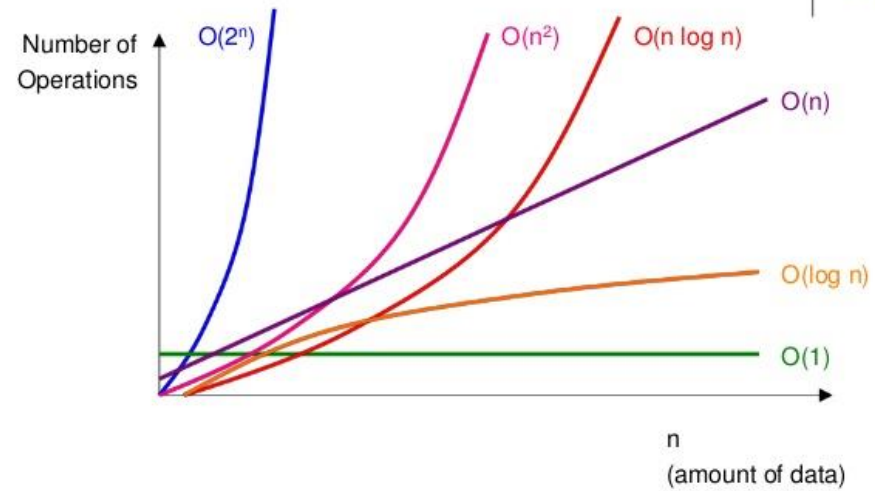
- 문제 조건이 $N=10$ 만이라면? $O(N)$ 이나 $O(N\log N)$

#시간 복잡도

<Time Complexity>

$$O(1) < O(\log N) < O(\sqrt{N}) < O(N) < O(N \log N) < O(N^2) < \dots < O(2^N) < O(N!)$$

Comparing Big O Functions



(C) 2010 Thomas J Cortina, Carnegie Mellon University



#누적 합

<Prefix Sum>

3 구간 합 구하기 4 (BOJ #11659)

<문제 설명>

- 수 N 개가 주어짐
- i 번째 수부터 j 번째 수까지 합을 구하기
- 합을 구하는 연산은 M 번 들어옴

<제약 조건>

- $1 \leq N \leq 100,000$
- $1 \leq M \leq 100,000$
- $1 \leq i \leq j \leq N$

#누적 합

<Prefix Sum>

3 구간 합 구하기 4 (BOJ #11659)

<문제 설명>

- 수 N 개가 주어짐
- i 번째 수부터 j 번째 수까지 합을 구하기
- 합을 구하는 연산은 M 번 들어옴

<제약 조건>

- $1 \leq N \leq 100,000$
- $1 \leq M \leq 100,000$
- $1 \leq i \leq j \leq N$

<시간 복잡도>

연산 횟수 M 이 최대 10만이기 때문에,
합을 구할 때 연산을 1,000번 이내로 해야 함

N 이 최대 10만이기엔,
연산 당 $O(\sqrt{N})$ 이하의 시간 복잡도가 필요

-> 누적 합을 이용하면 $O(1)$ 에 가능!

#누적 합

<Prefix Sum>

배열 P를 생각해보자: $P[n] = \sum_{i=1}^n A[i]$

i	0	1	2	3	4	5	6	7	8
A[i]	-	3	1	4	1	5	9	2	6

누적 합을 구현할 때는 1-base로 사용하는 게 구현이 편해요!

i	0	1	2	3	4	5	6	7	8
P[i]	0	3	4						

#누적 합

<Prefix Sum>

배열 P를 다음과 같이 생각해보면: $P[n] = \sum_{i=1}^n A[i]$

i	0	1	2	3	4	5	6	7	8
A[i]	-	3	1	4	1	5	9	2	6
P[i]	0	3	4	8	9	14	23	25	31

i 번째 원소부터 j 번째 원소까지의 합은, $P[j] - P[i - 1]$ 과 같습니다.

i	0	1	2	3	4	5	6	7	8
P[i]	0	3	4	8	9	14	23	25	31

4번째 수부터 7번째 수까지 합은 $P[7] - P[4-1]$ 으로 구할 수 있어요!

#누적 합

<Prefix Sum>

3 구간 합 구하기 4 (BOJ #11659)

<문제 설명>

- 수 N개가 주어짐
- i번째 수부터 j번째 수까지 합을 구하기
- 합을 구하는 연산은 M번 들어옴

<제약 조건>

- $1 \leq N \leq 100,000$
- $1 \leq M \leq 100,000$
- $1 \leq i \leq j \leq N$

```
int main() {
    int N, M, L, R;
    cin >> N >> M;
    vector<int> A(N+1), P(N+1);
    for(int i=1; i <= N; i++){
        cin >> A[i];
        P[i] += P[i-1]+A[i];
    }
    while(M--){
        cin >> L >> R;
        cout<<P[R]-P[L-1]<<'\n';
    }
}
```

#누적 합

<Prefix Sum>

3 구간 합 구하기 4 (BOJ #11659)

<시간복잡도 계산>

- 누적 합을 저장하는 P[] 전처리에 $O(N)$
- 구간 합을 구하는 연산 $O(1)$
- 구간 합을 구하는 횟수 M번
- 시간 복잡도: $O(N+M)$

```
int main() {  
    int N, M, L, R;  
    cin >> N >> M;  
    vector<int> A(N+1), P(N+1);  
    for(int i=1; i <= N; i++){  
        cin >> A[i];  
        P[i] += P[i-1]+A[i];  
    }  
    while(M--){  
        cin >> L >> R;  
        cout<<P[R]-P[L-1]<<'\n';  
    }  
}
```

#누적 합

<Prefix Sum> - 2차원 누적 합

1 구간 합 구하기 5 (BOJ #11660)

<문제 설명>

- $N \times N$ 개의 수가 $N \times N$ 크기의 표에 채워져 있음
- $(x1, y1)$ 부터 $(x2, y2)$ 까지 합을 구하기
 - (x, y) 는 x 행 y 열
- 합을 구하는 연산은 M 번 들어옴

<제약 조건>

- $1 \leq N \leq 1,024$
- $1 \leq M \leq 100,000$
- $1 \leq x1 \leq x2 \leq N$
- $1 \leq y1 \leq y2 \leq N$
- 표에 채워진 수는 1000 이하 자연수

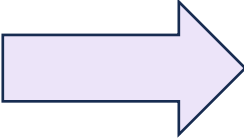
#누적 합

<Prefix Sum> - 2차원 누적 합

배열 P를 생각해보자: $P[n][m] = \sum_{i=1}^n (\sum_{j=1}^m A[i][j])$

A

1	2	3	4
2	3	4	5
3	4	5	6
4	5	6	7



P

1	3	6	10
3	8	15	24
6	15	27	42
10	24	42	64

#누적 합

<Prefix Sum> - 2차원 누적 합

$(x1, y1)$ 부터 $(x2, y2)$ 까지의 합: $P[x2][y2] - P[x1-1][y2] - P[x2][y1-1] + P[x1-1][y1-1]$

A

1	2	3	4
2	3	4	5
3	4	5	6
4	5	6	7

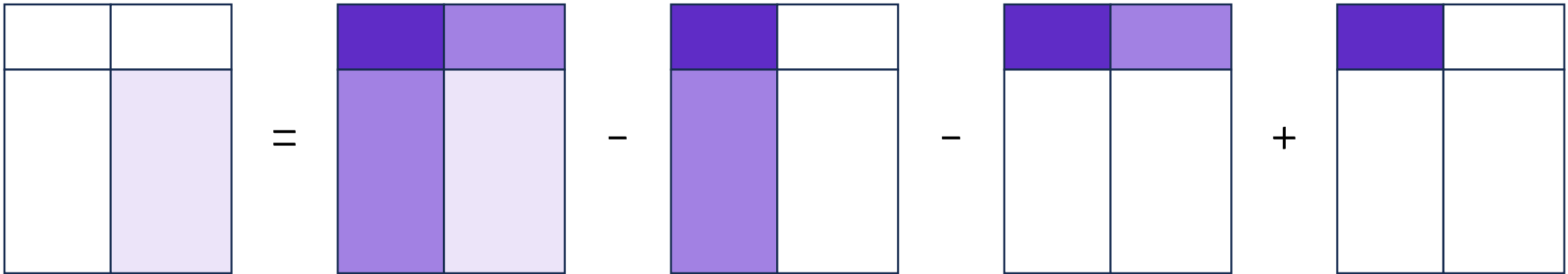
P

1	3	6	10
3	8	15	24
6	15	27	42
10	24	42	64

#누적 합

<Prefix Sum> - 2차원 누적 합

$(x1, y1)$ 부터 $(x2, y2)$ 까지의 합: $P[x2][y2] - P[x1-1][y2] - P[x2][y1-1] + P[x1-1][y1-1]$



#누적 합

<Prefix Sum> - 2차원 누적 합

Q: 그림 배열 P는 어떤 방법으로 초기 값을 설정하나요?

A: 전 슬라이드 식 $(x1, y1), (x2, y2)$ 에 (N, M) 을 넣어봅시다!

$$A[N][M] = P[N][M] - P[N-1][M] - P[N][M-1] + P[N-1][M-1]$$

정리하면...

$$P[N][M] = A[N][M] + P[N-1][M] + P[N][M-1] - P[N-1][M-1]$$

#누적 합

<Prefix Sum>

1 구간 합 구하기 5 (BOJ #11660)

<문제 설명>

- $N \times N$ 개의 수가 $N \times N$ 크기의 표에 채워져 있음
- $(x1, y1)$ 부터 $(x2, y2)$ 까지 합을 구하기
 - (x, y) 는 x 행 y 열
- 합을 구하는 연산은 M 번 들어옴

```

int main(void)
{
    ios::sync_with_stdio(0); cin.tie(0); // 빠른 입출력
    int A[1025][1025]={0,}, P[1025][1025]={0,};
    int N, M, x1, y1, x2, y2;
    cin >> N >> M;
    for(int i=1; i <= N; i++){
        for(int j=1; j <= N; j++){
            cin >> P[i][j];
            P[i][j] += P[i-1][j] + P[i][j-1] - P[i-1][j-1];
        }
    }
    for(int i=0; i < M; i++){
        cin >> x1 >> y1 >> x2 >> y2;
        cout << P[x2][y2]-P[x1-1][y2]-P[x2][y1-1]+P[x1-1][y1-1];
        cout << '\n';
    }
}

```



#필수 문제

3 구간 합 구하기 4 (BOJ #11659)

1차원 누적합을 구현해보아요

1 구간 합 구하기 5 (BOJ #11660)

2차원 누적합을 구현해보아요

#연습 문제 도전

1 인간-컴퓨터 상호작용 (BOJ #16139)

여러 가지 누적 합을 미리 전처리하기

4 체스판 다시 칠하기 2 (BOJ #25682)

배열에 어떤 정보를 저장해야 할까요?

4 부분합 (BOJ #1806)

배울 것이 많은 좋은 문제

5 문자열 일치도 (BOJ #34837)

2025 HCPC Beginner Div. E번 | Advanced Div. C번

4 직사각형과 쿼리 (BOJ #14846)

문제의 조건에 주목해 봅시다.

3 나머지 합 (BOJ #10986)

특정 조건을 만족하는 구간의 개수는?

#시간 복잡도

<Time Complexity> - TMI 모음

시간 복잡도는 Big- Ω 표기법과 Big- θ 표기법으로도 표현할 수 있습니다.

Big- Ω 표기법은 $f(x)$ 가 “최소” $g(x)$ 에 비례하는 수준으로 증가할 때,
Big- θ 표기법은 $f(x)$ 가 $g(x)$ 에 비례하는 수준으로 증가할 때 사용할 수 있습니다.

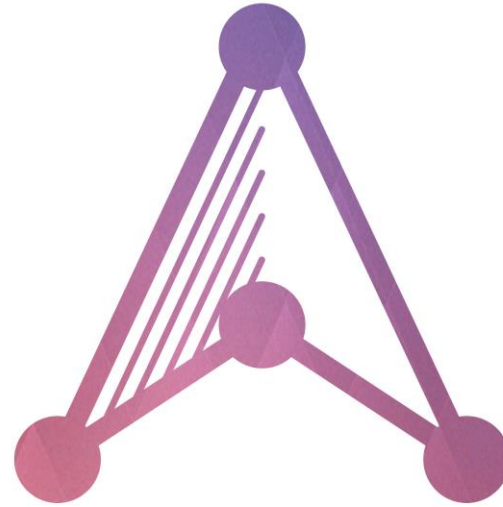
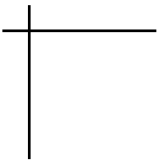
#시간 복잡도

<Time Complexity> - TMI 모음

$$\sum_{i=1}^N \frac{1}{i} = O(\log N)$$

$$\sum_{i=1}^N i^k = O(N^{K+1})$$

$$\log(N!) = O(N \log(N))$$



ALOHA
The algorithm club.

